LIBRARY MANAGEMENT SYSTEM – JAVA PROJECT REPORT

Course Name: Advanced Programming (OOPS)
Project Title: Library Management System
Student Name: Sohan Ghosh
Roll Number: 38

1. Problem Statement

Modern libraries face challenges in managing books efficiently. Manual record-keeping often leads to:

- Loss of book records
- Difficulty tracking issued and returned books
- Time-consuming search for book availability

Objective:
Develop a console-based **Library Management System** in Java using **OOP principles** to:

- Add, view, and manage books
- Store book records persistently
- Demonstrate inheritance, encapsulation, and file handling

Target Users:

- Librarians
- Students or library members

Scope:

- Maintain book catalog
- Track quantity and availability
- Implement persistent storage using file I/O

2. System Design

2.1 UML Class Diagram :

```
+----------------+
|     Book       |
+----------------+
| -id: int       |
| -title: String |
| -author: String|
| -quantity: int |
+----------------+
| +getId()       |
| +getTitle()    |
| +getAuthor()   |
| +getQuantity() |
+----------------+

+----------------+
|    Student     |
+----------------+
| -studentId:int |
| -name:String   |
+----------------+
| +getStudentId()|
| +getName()     |
+----------------+

+----------------+
|  IssueRecord   |
+----------------+
| -studentId:int |
| -bookId:int    |
+----------------+
| +getStudentId()|
| +getBookId()   |
+----------------+

+----------------+
|    Library     |
+----------------+
| -books:ArrayList<Book>|
| -FILE_NAME:String     |
+----------------+
| +addBook(Book)        |
| +viewBooks()          |
| +saveBooks()          |
| +loadBooks()          |
+----------------+
```

2.2 Major Classes Description :

- Book: Represents a single book with attributes like ID, title, author, quantity. Provides getters.
- Student: Represents library users (students) with ID and name.
- IssueRecord: Tracks which student has issued which book.
- Library: Core management class with ArrayList of books, file-based persistence, and methods to add/view books.

2.3 Package Structure :

 Book: Represents a single book with attributes like ID, title, author, quantity. Provides getters.

 Student: Represents library users (students) with ID and name.

 IssueRecord: Tracks which student has issued which book.

 Library: Core management class with ArrayList of books, file-based persistence, and methods to add/view books.

2.4 Interaction Flow :

1. User runs Main.java
2. Console menu appears
3. User chooses: Add Book / View Books / Exit
4. Library class methods are called accordingly
5. Books stored in memory and persisted in books.txt

3. OOP Concepts Demonstrated

3.1 Encapsulation :

- **Where it is used:** Book and Student classes.
- **Why it is used:** To hide internal data and provide controlled access through getters. This ensures that data like book ID or student ID cannot be modified directly by external code.
- **Code Snippet:**

```java
public class Book {
    private int id;
    private String title;
    public int getId() { return id; }
    public String getTitle() { return title; }
}
```

- **Explanation:** The id and title are private, so they cannot be changed directly. Access is provided only through public getter methods.

3.2 Abstraction :

- Where it is used: Library class methods like addBook() and viewBooks().
- Why it is used: Users interact with high-level methods without knowing the internal implementation (ArrayList operations, file handling).
- Code Snippet:

```
public void addBook(Book book){
    books.add(book);
    saveBooks();
}
```

Explanation: Users simply call addBook() to add a book. Internally, it updates the list and persists data in a file, abstracting complexity.

3.3 Inheritance :

- Where it could be used: Extending Book class for different book types (e.g., ReferenceBook, TextBook).
- Why it is used: To avoid code duplication and allow specialization.
- Code Snippet (Example):

```
public class ReferenceBook extends Book {
    private String subjectArea;
    public ReferenceBook(int id, String title, String author, int quantity, String subjectArea){
        super(id, title, author, quantity);
        this.subjectArea = subjectArea;
    }
}
```

Explanation: ReferenceBook inherits basic book attributes while adding subject-specific info.

3.4 Dynamic Data Structures (ArrayList) :

- Where it is used: Library class to store multiple books.
- Why it is used: Allows flexible addition/removal of books without a fixed-size array.
- Code Snippet:

```
private ArrayList<Book> books = new ArrayList<>();
```

Explanation: Books are stored in a dynamic list that grows as new books are added, improving scalability.

3.5 File I/O / Persistence :

- Where it is used: Library.saveBooks() and Library.loadBooks().
- Why it is used: Ensures book data persists across program runs.
- Code Snippet:

```
PrintWriter writer = new PrintWriter(new FileWriter("books.txt"));
for(Book b : books){
    writer.println(b.getId() + "," + b.getTitle() + "," + b.getAuthor() + "," + b.getQuantity());
}
writer.close();
```

Explanation: Saves all books to books.txt. When the program restarts, loadBooks() reads this file to rebuild the book list.

3.6 Exception Handling :

- Where it is used: Reading/writing books.txt.
- Why it is used: Prevents program crash if the file is missing or malformed.
- Code Snippet:

```
try {
    Scanner scanner = new Scanner(new File("books.txt"));
} catch (Exception e) {
    System.out.println("Error loading books.");
}
```

Explanation: Catches errors like FileNotFoundException, ensuring the program continues running smoothly.

3.7 Menu-Driven Console (User Interaction) :

- Where it is used: Main.java.
- Why it is used: Provides an intuitive interface for users to interact with the system.
- Code Snippet:

```
System.out.println("1. Add Book\n2. View Books\n3. Exit");
int choice = scanner.nextInt();
```

- Explanation: Users can navigate the system without knowing internal code; easy for non-technical users.

4. Implementation Highlights

4.1 Adding a Book :

```
Book book = new Book(id, title, author, quantity);
library.addBook(book);
```

Justification:

- Adds new book to ArrayList
- Calls saveBooks() to persist data

4.2 Viewing Books :

```
for(Book b : books){
    System.out.println(b.getId() + " | " + b.getTitle());
}
```

Justification:

- Iterates ArrayList to display book catalog
- Provides real-time view of available books

4.3 File-Based Persistence :

```
PrintWriter writer = new PrintWriter(new FileWriter("books.txt"));
for(Book b : books){
    writer.println(b.getId() + "," + b.getTitle() + "," + b.getAuthor() + "," + b.getQuantity());
}
writer.close();
```

Justification:

- Stores all book data in books.txt
- Ensures data is not lost on program exit

4.4 Loading Books :

```
Scanner scanner = new Scanner(new File("books.txt"));
while(scanner.hasNextLine()){
    String[] data = scanner.nextLine().split(",");
    books.add(new Book(Integer.parseInt(data[0]), data[1], data[2], Integer.parseInt(data[3])));
}
```

Justification:

- Reads stored book data at program start

- Rebuilds ArrayList in memory for operations

4.5 Menu-Driven Console :

```
System.out.println("1. Add Book\n2. View Books\n3. Exit");
int choice = scanner.nextInt();
```

Justification:

- Provides simple user interface
- Easy navigation for adding/viewing books

5. Testing & Error Handling

5.1 Test Cases Considered :

Feature: Add Book

- Test Case: Add a book with valid input (ID, title, author, quantity)
- Expected Result: The book is added to the library and stored in books.txt for persistence.

Feature: View Books

- Test Case 1: View books when the library contains books
- Expected Result: All stored books are displayed on the console with details (ID, title, author, quantity).
- Test Case 2: View books when the library is empty
- Expected Result: Program displays a message: “No books available.”

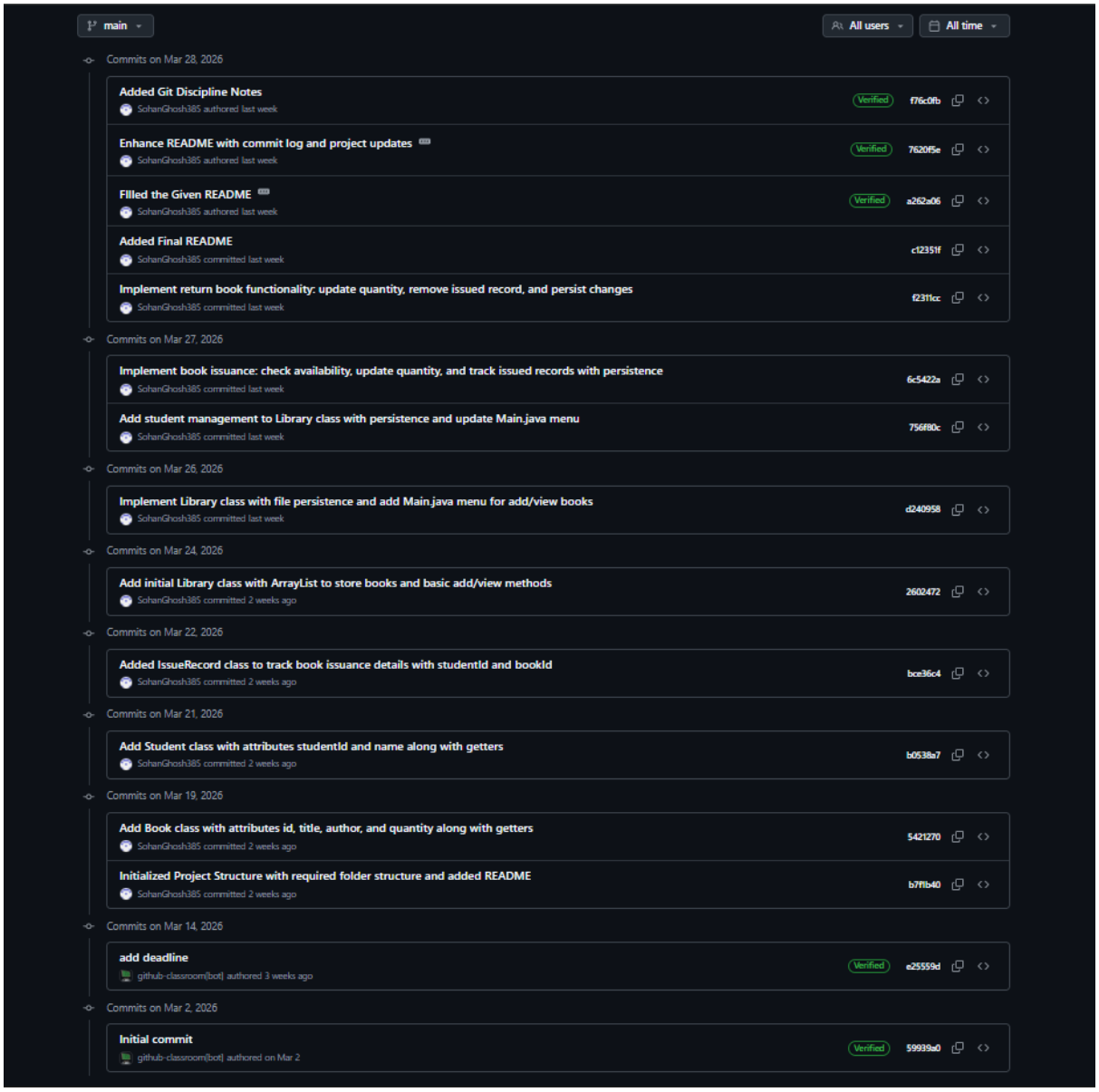
5.2 Edge Cases :

- Adding book with duplicate ID (currently allows duplicates)
- Empty or invalid input
- File books.txt missing or empty

5.3 Failure Scenarios Handled :

- File not found → handled using try-catch
- Parsing errors → handled to prevent program crash
- Invalid menu input → prints "Invalid choice"

6. Git Workflow



7. Conclusion & Future Scope

Conclusion :

- Developed a fully functional Library Management System in Java
- Demonstrated core OOP concepts: Encapsulation, Abstraction, ArrayList usage, File I/O
- Achieved persistent storage, menu-based console interaction, and exception handling

Future Scope :

- Add Book Issue/Return system with IssueRecord
- Implement user authentication for librarian and student roles
- Upgrade UI to JavaFX or Swing
- Add search functionality by title/author
- Convert file-based storage to SQLite or MySQL database